OADP

OpenShift API for Data Protection

Complete Beginner's Personal Notes

👤 About This Guide	
Audience :	Beginner
Level :	Beginner (basic Docker / Kubernetes / OpenShift assumed)
Course :	Red Hat OpenShift Administration III : DO380 Chapter 2
Sections :	14 topics covering ALL concepts in the course docs

Red Hat DO380 • Chapter 2: Backup, Restore and Migration of Applications with OADP

📋 Table of Contents

#	Topic	Page
1	Why Do We Need OADP ?	3
2	What Does OADP Back Up ? (The Three Layers)	6
3	Manual Export & Import — Before OADP	8
4	Backing Up Container Images	11
5	Backup Consistency — Inconsistent vs Crash vs App-Consistent	14
6	Volume Snapshots — CSI, VolumeSnapshotClass & VolumeSnapshot	16
7	OADP Architecture — Components & Overview	18
8	Object Storage & OpenShift Data Foundation Integration	21
9	Installing & Configuring OADP (DataProtectionApplication)	23
10	Backup & Restore — Resources, Lifecycle & Flow	27
11	Backup Hooks — Application-Consistent Backups	31
12	Scheduled Backups & Velero CLI Tool	35
13	File System Backup (FSB) — Volumes Without Snapshots	38
14	Troubleshooting, Commands & Cheat Sheet	40

1. Why Do We Need OADP?

Imagine your team's wiki application running on OpenShift stops working due to a server crash. Without a backup, everything — all your data, configurations, and code is gone. OADP is OpenShift's answer to this problem.

Scenario	What Could Go Wrong?	How OADP Helps
Server / disk crash	All persistent data destroyed	Restore from S3 backup in minutes
Accidental deletion	Admin deletes the namespace	Restore to previous working state
Cyberattack / ransomware	Data corrupted or encrypted	Restore from clean backup
Cluster migration	Moving to a new cluster	Export app and import to new cluster
Staging environment	Need copy of prod for testing	Restore to a new namespace
Pre-upgrade safety	Upgrading app — need rollback option	Snapshot before upgrade

1.1 The Three Things That MUST Be Backed Up

LAYER 1	LAYER 2	LAYER 3
Kubernetes Resources (App Blueprint) Deployments, Services,Secrets, Routes, PVCs...	Container Images (App Software) Images in OpenShiftinternal registry	Persistent Volume Data (App State) Database files, uploads,runtime data on disk

💡 Analogy: Think of It Like Saving a Game		
Your APPLICATION NAMESPACE	=	Your game character (level, equipment)
CONTAINER IMAGES	=	The game software itself (installed files)
PERSISTENT VOLUME DATA	=	Your save file (progress, achievements)
Lose any one of these and you can't fully restore the game to its previous state.		

1.2 Data Protection Solutions Available

OADP is Red Hat's built-in solution, but the OpenShift ecosystem also supports:

- Veeam Kasten for Kubernetes
- Storware Backup and Recovery
- IBM Storage Protect Plus
- Pure Storage Portworx Backup
- Catalogic CloudCasa
- Dell PowerProtect Data Manager

⚠️ **What OADP Does NOT Cover**
✗ etcd backup (the cluster database — separate procedure required)
✗ OpenShift Operator backups (operators must handle their own state)
✗ External services (e.g., Database-as-a-Service, external S3 buckets)
✗ Cluster infrastructure (nodes, networking, certificates)

2. What Does OADP Back Up ? (The Three Layers)

2.1 Layer 1: Kubernetes Resources

These are the YAML definitions that describe how your application is structured and configured:

Resource Type	What It Does
Deployment	Manages how many copies of a pod run and how to update them
StatefulSet	Like Deployment but for stateful apps (databases) — stable network IDs
Service	Internal network endpoint to reach your pods
Route	External URL to access your application from outside the cluster
Secret	Stores sensitive data (passwords, tokens) in base64 format
ConfigMap	Stores non-sensitive configuration data as key-value pairs
PersistentVolumeClaim (PVC)	Request for storage space — like a voucher for a hard disk
PersistentVolume (PV)	The actual storage resource bound to a PVC
Namespace	The project/container that holds all the above resources
ImageStream	Tracks versions (tags) of container images in the internal registry
BuildConfig	Defines how source code is built into a container image

2.2 Layer 2: Container Images

Images are stored in OpenShift's **internal image registry**. OADP (via the openshift plug-in) can back up and restore image streams. For manual export, tools like `skopeo` or `oc image mirror` are used.

2.3 Layer 3: Persistent Volume Data

This is the actual data stored on disk — database tables, uploaded files, application state. This is backed up using one of two methods:

Method	How It Works	When to Use
CSI Volume Snapshot	Point-in-time snapshot using storage driver's snapshot API	When storage supports snapshots (Ceph RBD, EBS, Azure Disk)
File System Backup (FSB)	Kopia copies files from the volume mount point to S3	When storage does NOT support snapshots (NFS, local)

📌 Critical Rule
For a complete backup of a stateful application you MUST include:
• persistentvolumeclaims (the PVC resource definition)
• persistentvolumes (the PV resource definition)
• namespace (to preserve UID/GID — explained in Section 11)
• pods (required for backup hooks to run — Section 11)

3. Manual Export & Import — Before OADP

Understanding the manual process helps you appreciate what OADP automates. Before OADP, admins exported applications by hand using oc commands.

3.1 Listing Application Resources

The `oc get all` command only shows a **subset** of resources — it misses Secrets and ConfigMaps. Always be explicit:

```
# List specific resource types you need
$ oc get deployment,svc,secret -n prod
NAME                    READY   UP-TO-DATE   AVAILABLE
deployment.apps/mysql      1/1     1            1
NAME          TYPE       CLUSTER-IP       PORT(S)
service/mysql   ClusterIP   172.30.210.241    9001/TCP
NAME                    TYPE    DATA
secret/mysql-credentials  Opaque  3
```

3.2 Exporting Resources

```
# Export any resource to YAML
$ oc -n prod get deployment/mysql -o yaml > backup_deployment.yaml

# The exported file contains runtime info that must be removed
# before importing to another namespace
```

3.3 Cleaning Exported YAML — What Must Be Removed

Field to Remove	Why Remove It?	Resource Type
metadata.namespace	Different namespace is the target	All resources
metadata.resourceVersion	Old cluster version ID — will conflict	All resources

metadata.creationTimestamp	Old timestamp — meaningless on import	All resources
metadata.generation	Runtime counter — auto-assigned	All resources
metadata.uid	Unique ID — auto-assigned on create	All resources
status.*	Runtime state — not applicable on import	All resources
spec.clusterIP / spec.clusterIPs	IP already allocated in source cluster	Services
spec.host	Hostname tied to source cluster's domain	Routes

3.4 Using yq to Clean YAML Automatically

The `yq` tool is a command-line YAML processor (like `jq` for JSON). The `del()` function removes specified fields:

```
# Clean a Service YAML (remove namespace and clusterIP)
$ yq 'del(.metadata.namespace) | del(.spec.clusterIP*)' \
  service-mysql.yaml > clean-service-mysql.yaml

# Clean a Route YAML (remove namespace and auto-assigned hostname)
$ yq 'del(.metadata.namespace) | del(.spec.host)' \
  route-frontend.yaml > clean-route-frontend.yaml

# Import cleaned resources
$ oc create -f clean-service-mysql.yaml -n prod-backup
service/mysql created
```

3.5 Resources You Do NOT Need to Recreate

Some resources are auto-created by OpenShift or by other resources — don't include them:

Resource	Who Creates It
ReplicaSet	Automatically created by Deployment — do NOT include
Endpoints	Automatically created by Service — do NOT include
Build	Automatically triggered by BuildConfig — do NOT include

ServiceAccount (default ones)	Namespace creates builder, deployer, default SAs automatically
ReplicationController	Created by DeploymentConfig — do NOT include

🤖 **What OADP Automates (vs Manual)**
✓ Determines correct restore ORDER automatically (Secrets before Pods, etc.)
✓ Strips all runtime fields automatically — no manual yq needed
✓ Skips managed resources (ReplicaSets, Endpoints) automatically
✓ Handles namespace remapping automatically with namespaceMapping
✓ Backs up 50+ resources in minutes — manual process takes hours

4. Backing Up Container Images

The OpenShift internal image registry stores images built inside the cluster (from BuildConfigs). These must be exported separately for a complete backup.

4.1 Exposing the Internal Registry

By default, the registry is **only accessible inside the cluster**. To use external tools like skopeo, a cluster-admin must expose it:

```
# Expose registry externally (cluster-admin required)
$ oc patch configs.imageregistry.operator.openshift.io/cluster \
  --patch '{"spec":{"defaultRoute":true}}' \
  --type merge

# This triggers an API server redeployment — can take ~10 minutes!

# Get the registry URL
$ REGISTRY=$(oc get route default-route \
  -n openshift-image-registry \
  --template '{{.spec.host}}')

# Non-admin users can get URL from any ImageStream
$ oc -n openshift get is/cli -ojsonpath="{.status.publicDockerImageRepository}{'\n'}"
```

4.2 Logging In to the Internal Registry

```
# Method 1: oc registry login (easiest — auto-detects everything)
$ oc registry login
Saved credentials for default-route-openshift-image-registry.apps.ocp4.example.com

# Method 2: podman login manually
$ podman login \
  -u $(oc whoami) \
  -p $(oc whoami -t) \
  --tls-verify=false \
  $REGISTRY
```

4.3 Image Export & Import Tools

Tool	Export Command	Best For
skopeo copy	skopeo copy docker://src docker://dst	Direct registry-to-registry copy
skopeo sync	skopeo sync --src docker --dest docker src dst	Copy ALL tags at once
oc image mirror	oc image mirror src:* dst	OpenShift-native (supports wildcards)
podman save/load	podman save img | bzip2 > img.tar.bz2	Local archive for air-gapped environments

```
# Export: Copy image from internal registry to remote registry
$ skopeo copy \
  docker://${REGISTRY}/project_name/imagestream:tag \
  docker://remote-registry.example.com/path/image:tag

# Export all tags with oc image mirror (wildcard *)
$ oc image mirror ${REGISTRY}/project_name/imagestream:* \
  remote-registry.example.com/path/image

# Export to local .tar archive
$ podman pull ${REGISTRY}/project/imagestream:tag
$ podman save ${REGISTRY}/project/imagestream:tag | bzip2 > image.tar.bz2

# Import: Load from archive and push to OpenShift
$ podman load -i image.tar.bz2
$ podman tag old-registry/app:1.2.3 ${REGISTRY}/newproject/app:1.2.3
$ podman push ${REGISTRY}/newproject/app:1.2.3
```

🔐 Required Roles
system:image-puller — pull images from a project's image streams
system:image-pusher — push images to a project's image streams
Project users/admins already have BOTH. The builder service account has image-pusher.

4.4 Automated Image Backup — Kubernetes Job

For automated in-cluster image backup, use a Job with the OpenShift CLI image:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: backup-image
  namespace: application
spec:
  template:
    spec:
      containers:
      - name: backup
        image: image-registry.openshift-image-registry.svc:5000/openshift/cli:latest
        command: ["/bin/bash", "-c"]
        args:
          - |
            oc registry login
            oc image mirror \
              image-registry.openshift-image-registry.svc:5000/app/myapp:* \
              file://myapp --dir /backup
        volumeMounts:
          - mountPath: /backup
            name: backup-pvc
```

5. Backup Consistency — Three Levels

When backing up data from a running app, the consistency level of the backup matters greatly:

Consistency Type	How It Works	Risk	Use Case
Inconsistent	Copy files while app writes freely	HIGH — data may be half-written ❌	Never recommended
Crash-consistent	Freeze disk I/O, take snapshot, resume	MEDIUM — disk state safe, RAM lost ⚠️	Default snapshot behavior
Application-consistent	App flushes memory to disk THEN snapshot	LOW — cleanest possible backup ✅	Best practice with hooks

💡 Analogy: Word Document Saves		
Inconsistent	=	Yanking the power cable while Word is mid-save
Crash-consistent	=	Force-quitting Word — the last saved state is on disk
App-consistent	=	Clicking File → Save ✓ then backing up — guaranteed clean

5.1 Hot Backup vs Cold Backup

Type	App Status During Backup	Downtime?	Method
Hot Backup (preferred)	Running (but quiesced)	None — hooks pause writes briefly	Hooks + Snapshots
Cold Backup	Stopped/scaled to 0	Yes — app unavailable	Scale down → copy → scale up

```
# Example: Cold backup procedure
$ oc scale deployment/myapp --replicas=0  # Stop app
# Take snapshot / copy data
$ oc scale deployment/myapp --replicas=1  # Restart app

# Example: MySQL hot backup quiescing
mysql> FLUSH TABLES WITH READ LOCK;      # Flush + lock (PRE hook)
# Take snapshot

mysql> UNLOCK TABLES;                    # Resume (POST hook)
```

5.2 Database-Specific Backup Tools

Database	Backup Tool	What It Does
MySQL / MariaDB	mysqldump / mariadb-dump	Export all tables to SQL file
PostgreSQL	pg_dump / CHECKPOINT	Export DB + flush to disk before snapshot
MongoDB	mongodump / db.fsyncLock()	Export + lock for consistent snapshot
Any database	Volume snapshot + hooks	Application-consistent without specialized tools

📌 Key Point

OADP achieves application-consistent backups through BACKUP HOOKS.

Without hooks, OADP only creates CRASH-CONSISTENT backups (snapshot-based).

Always verify that hooks ran correctly — check backup logs for hookPhase entries.

6. Volume Snapshots — CSI, Classes & Resources

A volume snapshot is a point-in-time copy of a persistent volume's state. It is fast (seconds), non-disruptive, and can be used to create new volumes with that exact data.

6.1 How Volume Snapshots Work

1	App uses a PersistentVolumeClaim (PVC) backed by a CSI storage driver
2	OADP requests a snapshot from the CSI driver via the Kubernetes Snapshot API
3	CSI driver creates a point-in-time snapshot of the volume (near-instant)
4	Snapshot is stored in the same storage backend as the original volume
5	Data Mover then uploads snapshot data to S3 for offsite protection

⚠️ **Not All Storage Drivers Support Snapshots**
Only CSI drivers that implement the snapshot interface can be used.
NFS and local storage do NOT support snapshots → use File System Backup (FSB) instead.
Always verify your storage class has a matching VolumeSnapshotClass.

6.2 CSI Drivers with Snapshot Support

Storage Provider	CSI Driver	Snapshot Support?
AWS Elastic Block Store	ebs.csi.aws.com	✅ Yes
Azure Disk	disk.csi.azure.com	✅ Yes
CephFS (Red Hat ODF)	openshift-storage.cephfs.csi.ceph.com	✅ Yes
Ceph RBD (Red Hat ODF)	openshift-storage.rbd.csi.ceph.com	✅ Yes
NetApp Trident	csi.trident.netapp.io	✅ Yes
NFS (subdir provisioner)	k8s-sigs.io/nfs-subdir-external-provisioner	❌ No — use FSB
Local storage	kubernetes.io/no-provisioner	❌ No — use FSB

6.3 VolumeSnapshotClass

Like **StorageClass** for PVCs, **VolumeSnapshotClass** configures which CSI driver handles snapshot creation. The driver name MUST match the StorageClass provisioner.

```
# List available VolumeSnapshotClasses
$ oc get volumesnapshotclasses
NAME                                DRIVER
ocs-storagecluster-cephfsplugin-snapclass   openshift-storage.cephfs.csi.ceph.com
ocs-storagecluster-rbdplugin-snapclass      openshift-storage.rbd.csi.ceph.com

# List StorageClasses to match drivers
$ oc get storageclasses
NAME                              PROVISIONER
ocs-external-storagecluster-cephfs      openshift-storage.cephfs.csi.ceph.com  ← MATCH
ocs-external-storagecluster-ceph-rbd    openshift-storage.rbd.csi.ceph.com     ← MATCH
nfs-storage                         k8s-sigs.io/nfs-subdir... ← NO MATCH = use FSB
```

6.4 Creating a VolumeSnapshot (Manual)

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: my-snapshot
  namespace: application   # MUST be same namespace as source PVC
spec:
  volumeSnapshotClassName: ocs-storagecluster-rbdplugin-snapclass
  source:
    persistentVolumeClaimName: application-data   # Source PVC

# Check if snapshot is ready (READYTOUSE must be 'true')
$ oc get volumesnapshot
NAME        READYTOUSE   SOURCEPVC         SNAPSHOTCONTENT
my-snapshot  true        application-data   snapcontent-798...cf6
```

6.5 Creating a PVC from a Snapshot (Restore)

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-snapshot-volume
  namespace: application   # Same namespace as snapshot
spec:
  storageClassName: ocs-external-storagecluster-ceph-rbd
  accessModes: [ReadWriteOnce]
  dataSource:
    apiGroup: snapshot.storage.k8s.io
    kind: VolumeSnapshot
    name: my-snapshot
  resources:
    requests:
      storage: 1Gi   # Must be >= snapshot size
```

7. OADP Architecture — Components & Overview

7.1 OADP Component Stack

OpenShift Cluster → openshift-adp namespace

▼ houses ▼

Component	Type	Role
OADP Operator / Controller	Deployment	Manages all OADP resources and reconciles DPA config
Velero	Deployment	Main backup/restore engine — exports/imports Kubernetes resources
Node Agent	DaemonSet (1 per node)	Runs file-system backup (FSB) — directly reads volume data on nodes

▼ uses tools ▼

Tool	What It Does	Where Data Goes
Kopia	Backup tool: encrypts, compresses, deduplicates volume data	S3 bucket (via backup repository)
CSI Snapshot API	Creates point-in-time disk snapshots using the storage driver	Storage backend (then moved to S3)
Data Mover	Moves snapshot content from cluster to S3	S3 bucket

7.2 Velero Plugins

Plug-in	Mandatory?	Purpose
openshift	✅ ALWAYS	Backs up OpenShift-specific resources (Routes, ImageStreams, registry images)
csi	✅ For snapshots	Enables CSI Snapshot API and Data Mover functionality
aws	✅ For S3 storage	Stores/retrieves backups on any S3-compatible storage
gcp	Optional	Google Cloud Storage + GCE Disk snapshots
azure	Optional	Azure Blob Storage + Azure Managed Disk snapshots
kubevirt	Optional	Virtual machine backups (OpenShift Virtualization)

7.3 OADP API Resources — All Objects You Work With

Resource Kind	Short Name	Lives In	Purpose
DataProtectionApplication	DPA	openshift-adp	Master OADP config — defines everything
BackupStorageLocation	BSL	openshift-adp	S3 bucket connection config (auto-created from DPA)
VolumeSnapshotLocation	VSL	openshift-adp	Cloud-native snapshot location (for non-CSI snapshots)
Backup	—	openshift-adp	Triggers a single backup operation
Restore	—	openshift-adp	Triggers a restore from an existing backup
Schedule	—	openshift-adp	Creates backups on a cron schedule
BackupRepository	BR	openshift-adp	Tracks Kopia repository per namespace (auto-managed)
VolumeSnapshot	VS	App namespace	Point-in-time disk snapshot (auto-created by OADP)
VolumeSnapshotClass	VSC	Cluster-wide	CSI driver config for creating snapshots

7.4 The Big Picture — Data Flow

📦	Your app lives in its own namespace — Deployments, PVCs, Secrets, etc.
⚙️	You create a Backup resource in openshift-adp namespace
📄	Velero exports all Kubernetes resource YAMLs to S3
📷	CSI plug-in takes volume snapshots (or Node Agent runs FSB)
🚚	Data Mover uploads snapshot/FSB data to S3 using Kopia (encrypted)
☁️	S3 bucket now has complete backup: Kubernetes YAMLs + volume data + logs

8. Object Storage & OpenShift Data Foundation

OADP requires object storage (S3-compatible) to store backups. In this course, OpenShift Data Foundation (ODF) provides this storage.

8.1 Object Storage Options

Provider	Type	Notes
AWS S3	Cloud	Native; direct use with aws plug-in
Google Cloud Storage	Cloud	Use gcp plug-in
Azure Blob Storage	Cloud	Use azure plug-in
MinIO	On-premise S3	S3-compatible; use aws plug-in with custom s3Url
ODF NooBaa MCG	On-cluster S3	Multi-cloud gateway; storageClass: openshift-storage.noobaa.io
ODF Ceph RGW	On-cluster S3	Ceph object gateway; storageClass: ocs-external-storagecluster-ceph-rgw

8.2 ObjectBucketClaim (OBC) — Requesting an S3 Bucket

An OBC is to S3 storage what a PVC is to block storage — it's a **request for an S3 bucket**:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: my-bucket-claim
  namespace: my-namespace
spec:
  storageClassName: openshift-storage.noobaa.io   # or ceph-rgw
  generateBucketName: my-bucket                   # prefix for name

# After a few minutes, check phase
$ oc get obc
NAME            STORAGE-CLASS                PHASE
my-bucket-claim  openshift-storage.noobaa.io   Bound  ← Ready!
```

8.3 Getting Bucket Credentials & Info

```
# ODF creates a ConfigMap with bucket connection info
$ oc get configmap/my-bucket-claim -o yaml
data:
  BUCKET_HOST: s3.openshift-storage.svc    # Internal URL
  BUCKET_NAME: my-bucket-9ce46e22-2fb8...  # Generated name
  BUCKET_PORT: '443'
  BUCKET_REGION: ''                        # Use 'us-east-1' if empty

# ODF creates a Secret with credentials
$ oc extract secret/my-bucket-claim --to -
# AWS_ACCESS_KEY_ID
YEAsbMJnG3o1bGANZprt
# AWS_SECRET_ACCESS_KEY
xjaeCDhskn7lfrdA7WqzoUxpiRYuyjc9uDaWlMw3

# For external access, get the S3 route URL
$ oc get route/s3 -n openshift-storage -o jsonpath='{.spec.host}{"\n"}'
s3-openshift-storage.apps.ocp4.example.com
```

8.4 Validating S3 with s3cmd

```
# Configure s3cmd (~/.s3cfg)
access_key = AWS_ACCESS_KEY_ID
secret_key = AWS_SECRET_ACCESS_KEY
host_base = s3.openshift-storage.svc
host_bucket = s3.openshift-storage.svc/%(bucket)s
signature_v2 = True   # Required for ODF

# Test connection
$ s3cmd ls          # List all buckets
$ s3cmd la          # List all objects in all buckets
$ s3cmd ls s3://my-bucket/oadp/   # Browse backup directory
```

9. Installing & Configuring OADP

9.1 Installation Steps

1	Install OADP Operator via OperatorHub (web console or oc CLI)
2	Create an S3 bucket (ObjectBucketClaim or external cloud bucket)
3	Retrieve S3 credentials (oc extract secret/obc-name --to -)
4	Create a credentials file and Velero secret (cloud-credentials)
5	Create the DataProtectionApplication (DPA) YAML and apply it
6	Verify BackupStorageLocation enters 'Available' phase

⚠️ **Critical Installation Order**
The Velero secret (cloud-credentials) MUST EXIST before creating the DPA.
Creating DPA first → installation fails → you must delete and recreate DPA.
Always: Secret first → DPA second.

9.2 Creating the Velero Secret

```
# credentials-velero file format
[default]                    # Profile name — must match DPA config
aws_access_key_id=YEAsbMJnG3o1bGANZprt
aws_secret_access_key=xjaeCDhskn7lfrdA7WqzoUxpiRYuyjc9uDaWlMw3

# Create the secret in openshift-adp namespace
$ oc create secret generic cloud-credentials \
  -n openshift-adp \
  --from-file cloud=credentials-velero
```

9.3 DataProtectionApplication — Complete Annotated YAML

```yaml
apiVersion: oadp.openshift.io/v1alpha1
kind: DataProtectionApplication
metadata:
  name: oadp-backup
  namespace: openshift-adp
spec:
  # ── COMPONENT CONFIGURATION ──────────────────────────────
  configuration:
    velero:
      defaultPlugins:
        - aws       # S3-compatible storage
        - openshift  # MANDATORY — OpenShift resources
        - csi       # MANDATORY — Volume snapshots + Data Mover
      defaultSnapshotMoveData: true  # Enable Data Mover (snapshot → S3)
    nodeAgent:
      enable: true          # Required for FSB and Data Mover
      uploaderType: kopia    # Required for Data Mover (not restic)
      podConfig:            # Optional: resource limits per node-agent pod
        resourceAllocations:
          limits:
            cpu: '1'
            memory: 8Gi
          requests:
            cpu: 500m
            memory: 256Mi
  # ── BACKUP STORAGE LOCATION (where backups are stored) ─
  backupLocations:
    - velero:
        provider: aws            # Use for any S3-compatible storage
        default: true            # This BSL is used if none specified
        credential:
          key: cloud             # Key within the secret
          name: cloud-credentials # Name of the secret
        config:
          profile: 'default'      # Profile name in credentials file
          region: 'us-east-1'     # Required even for non-AWS (set any value)
          s3Url: https://s3.openshift-storage.svc  # Custom endpoint
          s3ForcePathStyle: 'true'  # Required for ODF/MinIO
          insecureSkipTLSVerify: 'true'  # Skip TLS; needed for ImageStream backup
        objectStorage:
          bucket: my-bucket-9ce46e22-2fb8-4a46-af95-f6949d87c3fd
          prefix: oadp           # Subdirectory inside bucket
```

9.4 Key DPA Fields Explained

Field	What It Does
defaultPlugins	List of Velero plug-ins to load — always include openshift and csi
defaultSnapshotMoveData: true	Automatically move all CSI snapshots to S3 via Data Mover
nodeAgent.enable: true	Required for FSB and Data Mover — deploys node-agent DaemonSet
uploaderType: kopia	Data Mover REQUIRES kopia (not restic); use kopia for all cases
provider: aws	Use for ANY S3-compatible storage (not just AWS)
s3Url	Custom S3 endpoint URL (for ODF, MinIO — omit for real AWS S3)
s3ForcePathStyle: true	Use path-style URLs (https://host/bucket) — required for ODF/MinIO
insecureSkipTLSVerify: true	Skip TLS cert verification — also required for ImageStream backup
caCert	Base64 CA cert for custom TLS — NOT compatible with ImageStream backup
prefix	Subdirectory in bucket for OADP data (keeps bucket organized)
ttl	Time-to-live in backup template — default 720h (30 days), minimum 1h

9.5 Verifying OADP Installation

```
# Verify all OADP components are running
$ oc -n openshift-adp get deploy
NAME                          READY
openshift-adp-controller-manager   1/1   ← OADP operator
velero                        1/1   ← Velero backup engine

$ oc -n openshift-adp get daemonset
NAME        DESIRED   CURRENT   READY
node-agent   3        3         3     ← one per compute node

# CRITICAL: Verify backup storage location is reachable
$ oc -n openshift-adp get backupstoragelocation
NAME          PHASE       LAST VALIDATED
oadp-backup-1  Available   7s   ← Must be 'Available'!

# If 'Unavailable', check the error:
$ oc -n openshift-adp describe backupstoragelocation oadp-backup-1
Status:
  Message: InvalidAccessKeyId: The AWS access key Id does not exist…
```

10. Backup & Restore — Resources, Lifecycle & Flow

10.1 Backup Resource — Full YAML

```yaml
apiVersion: velero.io/v1
kind: Backup
metadata:
  name: my-app-backup
  namespace: openshift-adp   # All OADP resources live here
spec:
  includedNamespaces:      # Which projects to back up
  - my-app-project
  ttl: 720h0m0s            # Keep for 30 days (default: 30d, min: 1h)
  labelSelector:           # Back up resources with THIS label
    matchLabels:
      app: my-app
  orLabelSelectors:        # Back up resources matching ANY of these labels
  - matchLabels:
      app: my-app
  - matchLabels:
      kubernetes.io/metadata.name: my-app-project  # Namespace itself
  includedResources:       # Explicit list of resource types to include
  - deployments
  - statefulsets
  - services
  - routes
  - secrets
  - persistentvolumeclaims
  - persistentvolumes
  - pods                   # REQUIRED for hooks to run
  - namespace              # REQUIRED for UID/GID preservation
```

10.2 Backup Status Lifecycle

New	Backup created — validation in progress
FailedValidation	YAML definition has errors — check validationErrors field
InProgress	Backup actively running — Velero exporting resources
WaitingForPluginOperations	Data Mover uploading snapshot data to S3
WaitingForPluginOperationsPartiallyFailed	Data Mover upload partially failed
Finalizing	Saving logs, results, metadata to S3 — almost done
FinalizingPartiallyFailed	Finalization with some resource failures
Completed ✅	ALL data in S3 — backup is ready and usable for restore
PartiallyFailed ⚠️	Some resources failed — may still be partially restorable
Failed ❌	Backup cannot be used for restore — investigate and retry

10.3 Checking Backup Status

```
# Quick status check
$ velero get backup
NAME          STATUS    ERRORS   WARNINGS   EXPIRES   STORAGE
my-app-backup   Completed  0       0        29d      oadp-backup-1

# Detailed info (reads from S3 when --details is used)
$ velero describe backup my-app-backup --details
Phase:  Completed
Namespaces: Included: my-app-project
Resources:  Included: deployments, services, routes...
Items backed up: 7 / 7

Resource List:
  apps/v1/Deployment: - my-app-project/frontend
  v1/Service:         - my-app-project/frontend
  ...
```

10.4 Restore Resource — Full YAML

```yaml
# Restore to same namespace
apiVersion: velero.io/v1
kind: Restore
metadata:
  name: my-app-restore
  namespace: openshift-adp
spec:
  backupName: my-app-backup   # Name of the Backup to restore from

# Restore to DIFFERENT namespace (staging/migration)
apiVersion: velero.io/v1
kind: Restore
metadata:
  name: wiki-staging
  namespace: openshift-adp
spec:
  backupName: wiki
  namespaceMapping:
    wiki: wiki-staging       # source-ns: destination-ns
```

10.5 Important Restore Behaviors

Behavior	Detail
Only missing resources restored	Resources already in target namespace are SKIPPED (not overwritten)
Labels added to restored resources	velero.io/restore-name and velero.io/backup-name are set on all resources
Auto-excluded resources	nodes, events, Velero CRDs, CSI attachments — auto-excluded
Namespace required for UID/GID	Include namespace resource to preserve security context IDs
Avoid managed resources	Don't include ReplicaSets, Endpoints, Builds — they're auto-created

10.6 Object Storage Layout After Backup

```
s3://my-bucket/
├── docker/                    ← Container images (Docker v2 registry format)
│   └── registry/v2/repositories/
│       └── wiki/hugo/
└── oadp/
    ├── backups/               ← Kubernetes resource YAMLs + metadata
    │   └── my-app-backup/
    │       ├── my-app-backup.tar.gz        ← All Kubernetes YAMLs
    │       ├── my-app-backup-logs.gz       ← Velero backup logs
    │       └── my-app-backup-results.gz    ← Summary of backed-up items
    ├── kopia/                 ← Volume data (encrypted, per namespace)
    │   └── my-app-project/
    │       ├── kopia.repository            ← Kopia repo metadata
    │       └── (encrypted volume blocks)
    └── restores/
        └── my-app-restore/
            └── restore-my-app-restore-logs.gz
```

🔄 S3 Synchronization Behavior
OADP continuously syncs backup resources between S3 and the cluster.
If backup exists in cluster but NOT in S3 → OADP DELETES the cluster resource
If backup exists in S3 but NOT in cluster → OADP CREATES the cluster resource
ONLY 'Completed' backups are synced. Failed/PartiallyFailed are NOT auto-synced.
This is why you MUST use 'velero delete', not 'oc delete' to remove backups!

11. Backup Hooks — Application-Consistent Backups

Hooks let you run commands **inside application pods** before and after backup operations. This is the key to achieving application-consistent backups without downtime.

💡 The Hook Pattern
PRE HOOK → Tell app: 'Stop writing, prepare for snapshot'
SNAPSHOT → Point-in-time disk photo taken (seconds)
POST HOOK → Tell app: 'Resume normal operations'
Result: A perfectly clean, consistent backup with minimal (seconds) impact.

11.1 All Four Hook Types

Hook	When It Fires	Use Case	If It Fails
pre backup	BEFORE snapshot creation	Quiesce app, flush writes, lock DB, create lock file	Backup stops → Failed status
post backup	AFTER snapshot completes	Resume app, unlock DB, remove lock file	Backup stops → PartiallyFailed
init restore	AFTER restore, BEFORE containers start	Remove lock files, prep restored state (init container)	Restore continues silently — check pod status!
post restore	AFTER restored pod is running	Integrity check on restored data	Error logged — restore continues

⚠️ Hooks are SILENTLY IGNORED if pods is not in includedResources
You MUST add 'pods' to includedResources for hooks to execute.
If pods are missing, OADP skips hooks without any error or warning.
Always verify hook execution by checking backup logs for 'hookPhase' entries.

11.2 Complete Backup with Hooks — MongoDB Example

```yaml
apiVersion: velero.io/v1
kind: Backup
metadata:
  name: mongodb
  namespace: openshift-adp
spec:
  includedNamespaces: [mongodb]
  orLabelSelectors:
  - matchLabels: {app: mongodb}
  - matchLabels: {kubernetes.io/metadata.name: mongodb}  # namespace
  includedResources:
  - deployments
  - services
  - secret
  - pvc
  - pv
  - pods        # REQUIRED for hooks
  - namespace   # REQUIRED for UID/GID preservation
  hooks:
    resources:
    - name: mongodb-lock
      labelSelector:
        matchLabels: {app: mongodb}
      pre:
      - exec:
          container: mongodb
          command: [/usr/bin/mongosh, --eval, 'db.fsyncLock();']
      post:
      - exec:
          container: mongodb
          command: [/usr/bin/mongosh, --eval, 'db.fsyncUnlock();']
```

11.3 Restore with Init Hook — MongoDB Lock Removal

```yaml
apiVersion: velero.io/v1
kind: Restore
metadata:
  name: mongodb
  namespace: openshift-adp
spec:
  backupName: mongodb
  hooks:
    resources:
    - name: remove-lock
      labelSelector:
        matchLabels: {app: mongodb}
      postHooks:
      - init:
          initContainers:
          - name: remove-db-lock
            image: mongodb/mongodb-community-server:7.0-ubi9
            volumeMounts:
            - name: mongodb-data
              mountPath: /data/db
            command: [/usr/bin/rm, /data/db/mongod.lock]
```

11.4 UID/GID Preservation — Why Include the Namespace Resource

OpenShift assigns **unique UIDs and GIDs** to each namespace. App pods run with these IDs and write data to PVs owned by those IDs.

If you restore to a new namespace without including the `namespace` resource, the new project gets DIFFERENT UIDs/GIDs → pods can't read their own restored data (permission denied).

Action	Result
Include namespace in backup + restore	New namespace inherits original UID/GID → pods can access data ✅
Don't include namespace	New namespace gets new UID/GID → pods get permission errors ❌

11.5 Lab Example: MediaWiki + PostgreSQL Hooks

```
 MediaWiki PRE hook — create lock file (read-only mode)
container: mediawiki
command: [/bin/bash, -c, 'echo "backup in progress" > /data/images/lock_yBgMBwiR']

# MediaWiki POST hook — remove lock file (resume writes)
container: mediawiki
command: [/bin/bash, -c, 'rm -f /data/images/lock_yBgMBwiR']

# PostgreSQL PRE hook — flush all memory to disk
container: postgresql
command: [/bin/bash, -c, "psql -c 'CHECKPOINT;'"]

# Restore INIT hook — remove lock file before app starts
command: [/bin/rm, -f, /data/images/lock_yBgMBwiR]
```

12. Scheduled Backups & Velero CLI Tool

12.1 Cron Format — Quick Reference

```
# ┌──── minute  (0-59)
# │  ┌── hour    (0-23, 0=midnight)
# │  │  ┌ day-of-month (1-31)
# │  │  │  ┌ month (1-12)
# │  │  │  │  ┌ day-of-week (0-7, 0/7=Sunday)
# m  h  d  M  W
  0  23 *  *  *   ← Every day at 11 PM (23:00)
  0  7  *  *  *   ← Every day at 7 AM
  0  0  *  *  0   ← Every Sunday midnight
  30 6  *  *  1-5 ← Mon-Fri at 6:30 AM
  0  */6 * *  *   ← Every 6 hours
```

12.2 Schedule Resource YAML

```yaml
apiVersion: velero.io/v1
kind: Schedule
metadata:
  name: website-daily
  namespace: openshift-adp
  labels:
    app: hugo        # Labels inherited by all backup resources created
spec:
  schedule: '0 23 * * *'   # Every day at 11 PM
  paused: false            # true = disabled (prevents auto-backups)
  template:                # Same spec as a Backup resource
    includedNamespaces: [website]
    labelSelector:
      matchLabels: {app: hugo}
    includedResources:
    - imagestreams
    - buildconfigs
    - deployments
    - services
    - routes
    ttl: 720h0m0s          # Each backup auto-deleted after 30 days
```

12.3 Triggering an On-Demand Backup from a Schedule

```
# Trigger immediate backup using existing schedule as template
$ velero create backup pre-upgrade-1.1 \
  --from-schedule website-daily
Backup request 'pre-upgrade-1.1' submitted successfully.

# List backups with inherited labels
$ velero get backup -l app=hugo
NAME                        STATUS     ERRORS
pre-upgrade-1.1             Completed  0
website-daily-20251215100856  Completed  0
website-daily-20251215091728  Completed  0
```

12.4 The Velero CLI Tool

OADP includes the `velero` CLI tool inside the Velero pod. Set up an alias to use it:

```
# Set up velero alias (run once per session)
$ alias velero='oc -n openshift-adp exec deployment/velero -c velero -it -- ./velero'

# velero vs oc — same info, different detail level
$ oc -n openshift-adp get backup,restore    # Basic Kubernetes view
$ velero get backup                         # Richer status view
$ velero get restore
$ velero get schedule

# Create resources from CLI
$ velero create restore website-dev \
  --from-backup website-label \
  --namespace-mappings website:website-dev
```

12.5 Deleting Backups — The Right Way

```
# WRONG — oc delete will be undone by S3 sync!
$ oc -n openshift-adp delete backup my-app-backup  ← BAD

# CORRECT — velero delete removes from both cluster AND S3
$ velero delete backup my-app-backup
Are you sure? y
Request to delete backup submitted. Status changes to 'Deleting'.

# Note: S3 data deletion is ASYNCHRONOUS
# Kopia repository data may take up to 24h to be fully purged
# OADP runs periodic repository maintenance to reclaim space
```

📌 Backup TTL — Auto-Deletion
ttl: 720h0m0s = 30 days (default) — minimum is 1 hour
After TTL expires: backup is deleted from both OpenShift cluster AND S3
Schedules can use labels to track which backups belong to which app
Use 'velero get backup -l app=myapp' to list app-specific backups

13. File System Backup (FSB) — Volumes Without Snapshots

File System Backup (FSB), also called **Pod Volume Backup**, copies volume data directly from the file system — useful when the storage driver doesn't support CSI snapshots.

13.1 How FSB Works

1	Admin creates Backup with FSB enabled (flag or annotation)
2	Velero exports Kubernetes resources to S3 (same as always)
3	Node Agent DaemonSet (running on same node as app pod) reads the volume's mount point on the node file system
4	Kopia compresses and uploads the file data to the S3 backup repository

13.2 FSB Limitations

Limitation	Detail
hostPath volumes	FSB does NOT support hostPath volumes at all
App can write during backup	Files may change → use backup hooks to quiesce
Slower than snapshots	Copies all files vs snapshot's pointer approach
App pod must be running	Node Agent accesses data via the pod's mount point

13.3 Enabling FSB — Method 1: Entire Backup

```
apiVersion: velero.io/v1
kind: Backup
metadata:
  name: my-fsb-backup
  namespace: openshift-adp
spec:
  defaultVolumesToFsBackup: true   # ALL volumes use FSB
```

13.4 Enabling FSB — Method 2: Per-Volume Annotation

When an app uses **mixed storage** (some volumes support snapshots, others don't), annotate the pod to specify which volumes use FSB:

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: website-nginx}
spec:
  template:
    metadata:
      annotations:
        # Comma-separated list of volume names to back up with FSB
        backup.velero.io/backup-volumes: wwwdata
        # Multiple: backup.velero.io/backup-volumes: vol1,vol2,vol3
    spec:
      volumes:
      - name: wwwdata
        persistentVolumeClaim: {claimName: nginx-wwwdata}
```

13.5 Snapshot vs FSB — Decision Guide

Situation	Recommended Method
Storage has CSI snapshot support (Ceph RBD, EBS)	CSI Snapshot + Data Mover ✅
Storage has NO snapshot support (NFS, local)	File System Backup (FSB) ✅
Mixed volumes — some snap, some don't	Annotation-based FSB for non-snap volumes ✅
hostPath volumes	Neither supported — use different storage ❌
App pod not running	FSB won't work — pod must be up for Node Agent to access data

📌 FSB + Data Mover Use Same S3 Repository
Both FSB and Data Mover store data in the same Kopia repository on S3.
OADP creates one Kopia repository per namespace under oadp/kopia/<namespace>/
Data is always encrypted, compressed, and deduplicated regardless of method.

14. Troubleshooting, Commands & Cheat Sheet

14.1 Troubleshooting Decision Tree

Symptom	Likely Cause	Fix
BSL stuck in Unavailable	Wrong S3 credentials or wrong URL	Fix credentials-velero, recreate secret, delete+recreate DPA
Backup: FailedValidation	YAML syntax error or invalid field	oc describe backup <n> → check validationErrors
Backup: PartiallyFailed	A hook failed or resource couldn't be backed up	velero backup logs <n> | grep -i error
Hooks not running	'pods' not in includedResources	Add pods to includedResources in Backup YAML
Hook command failing	Wrong command, missing tool, wrong credentials	velero backup logs <n> | grep hookPhase
Volume not in backup	CSI snapshot not available or FSB not enabled	Add annotation or use defaultVolumesToFsBackup: true
Data inaccessible after restore	UID/GID mismatch in new namespace	Include 'namespace' resource in backup
Restore skips resources	Resources already exist in target namespace	Expected behavior — OADP skips existing resources
Backup recreates after oc delete	S3 sync recreates it	Always use 'velero delete backup <n>'
node-agent not running	nodeAgent.enable: false in DPA	Set nodeAgent.enable: true and update DPA
READYTOUSE=false on snapshot	CSI driver issue or snapshot class mismatch	Check VSC driver matches StorageClass provisioner

14.2 Complete Command Reference

Task	Command
Setup velero alias	alias velero='oc -n openshift-adp exec deployment/velero -c velero -it -- ./velero'
List backups	velero get backup
List restores	velero get restore
List schedules	velero get schedule

Backup detail (from cluster)	velero describe backup <name>
Backup detail (includes S3 data)	velero describe backup <name> --details
View backup logs	velero backup logs <name>
Search hook failures	velero backup logs <name> | grep hookPhase
Restore detail	velero describe restore <name> --details
Create backup from schedule	velero create backup <name> --from-schedule <schedule>
Create restore + namespace map	velero create restore <name> --from-backup <b> --namespace-mappings src:dst
Delete backup (cluster + S3)	velero delete backup <name>
Check BSL status	oc -n openshift-adp get backupstoragelocation
Check OADP pods	oc -n openshift-adp get deploy,daemonset
List VolumeSnapshotClasses	oc get volumesnapshotclasses
List StorageClasses	oc get storageclasses
Get OBC credentials	oc extract secret/<obc-name> --to -
Get OBC bucket info	oc get configmap/<obc-name> -o yaml
Validate S3 connection	s3cmd ls
List all S3 objects	s3cmd la
Download backup logs from S3	s3cmd get s3://bucket/oadp/backups/<n>/<n>-logs.gz
View all backups for an app	velero get backup -l app=<label>
Expose internal registry	oc patch configs.imageregistry.operator.openshift.io/cluster --patch '{"spec":{"defaultRoute":true}}' --type merge
Login to internal registry	oc registry login
Check OBC binding	oc get obc
Create Velero secret	oc create secret generic cloud-credentials -n openshift-adp --from-file cloud=credentials-velero

14.3 Key Concepts Quick Reference

Term / Acronym	Full Form & Meaning
OADP	OpenShift API for Data Protection — the backup operator
DPA	DataProtectionApplication — OADP's main config resource
BSL	BackupStorageLocation — S3 bucket connection config
VSL	VolumeSnapshotLocation — cloud-native snapshot storage config
CSI	Container Storage Interface — Kubernetes storage driver standard
FSB	File System Backup — Kopia-based backup for non-snapshot volumes
Data Mover	OADP component that moves snapshot data to S3 via Kopia
OBC	ObjectBucketClaim — request an S3 bucket from ODF (like PVC for object storage)
ODF	OpenShift Data Foundation — Red Hat's storage solution (Ceph + NooBaa)
MCG	MultiCloud Gateway (NooBaa) — S3-compatible storage from ODF
Ceph RGW	RADOS Object Gateway — Ceph's S3-compatible object storage
TTL	Time To Live — how long a backup is kept before auto-deletion (default 30d)
FSB annotation	backup.velero.io/backup-volumes: volname — marks specific volumes for FSB
namespaceMapping	In Restore YAML: redirects restore to different namespace
orLabelSelectors	Match resources with ANY label in the list (vs. all must match)
hookPhase	Log field to search for hook execution results (pre/post)
BackupRepository	Kopia repository tracking resource — one per namespace, auto-managed

14.4 Testing Checklist

Test	How to Verify	Pass Criteria
Backup completes	velero get backup	Status=Completed, Errors=0, Warnings=0
All resources backed up	velero describe backup --details → Resource List	All expected resource types present
Volume data backed up	Check S3 oadp/kopia/<ns>/ path	kopia.repository file exists
Hooks ran successfully	velero backup logs | grep hookPhase	hookPhase=pre and post present, no exit code 1
Restore completes	velero describe restore --details	Status=Completed, Items Restored matches expected
App works after restore	Access application URL	Application loads, data intact
Namespace mapping works	oc get all -n <target-ns>	Resources created in correct namespace
Init restore hook ran	Check pod logs / verify lock file removed	App pod Running (not Init:Error)
Schedule triggers on time	velero get schedule → LAST BACKUP column	Backup timestamp matches schedule
Old backups auto-deleted	velero get backup after TTL expires	Backup removed from cluster and S3
velero delete cleans S3	s3cmd ls after velero delete	Backup directory removed from S3 (allow 24h)
BSL always Available	oc get backupstoragelocation	Phase=Available, Last Validated recent

🎯 Top 8 Tips

1. Always verify BSL is 'Available' before running ANY backup
2. Use 'velero describe --details' for complete backup info (reads from S3)
3. Add 'pods' to includedResources or hooks will silently do nothing
4. Add 'namespace' to includedResources for stateful apps (UID/GID)
5. Always use 'velero delete', never 'oc delete' for backups
6. PartiallyFailed != totally broken — investigate warnings carefully
7. Test restores to staging BEFORE relying on backups for production
8. S3 cleanup after velero delete is async — allow up to 24 hours

More content adding in progress……

End of OADP Personal notes • DO380 Chapter 2

Red Hat OpenShift Administration III